Interaction summary through GigaChat (FT-6)

English version of ../AI.md. The specification allows applying AI to university data. In RTK CRM this is a single feature: a summary for an interaction card. A KAM opens the card and gets a short text — which stage the university is at, what has been done, what needs attention and what the next step is. The goal is practical: restoring the context before a call without rereading the whole thread.

What goes into the summary

Field	Source
University	`University.name`
Product	`ITProduct.name`
Stage	`WorkflowStageRef.name`
Scope	`Interaction.scope` (B2B or B2C)
Responsible KAM	`User.full_name`
Tasks	`Action.title`, `is_completed`, `due_date`
Comments	`Comment.text`

Personal data never reaches the prompt. SNILS, passport data, addresses and phone numbers are not stored in the card — they are dropped at the import stage (see `docs/SECURITY.md`). Only names, comment texts and service fields are passed into the prompt.

How it works

The request runs in two steps, as GigaChat requires:

1. The authorization key (`GIGACHAT_CREDENTIALS`) is sent to OAuth (`/api/v2/oauth`) in the `Authorization: Basic` header. The response carries an access token valid for 30 minutes. The `RqUID` header is mandatory — without it the service answers 400.
2. The model request (`/chat/completions`) goes with `Authorization: Bearer`.

The token is cached in process memory and refreshed ahead of time, minutes before expiry. One token serves all requests, so OAuth is not called for every summary.

The model response is limited to 150 words at temperature 0.3: what is needed is a summary of the facts in the card, not creative text. The system instruction explicitly forbids adding information that is not in the data.

Configuration

At minimum a single key is enough:

```
GIGACHAT_CREDENTIALS=<authorization-key-base64>
```

The Ministry of Digital Development CA certificate

GigaChat presents a certificate issued by the Ministry of Digital Development CA. Its root is absent from the default trusted certificate set of Python and most Linux images, so without it the TLS connection fails with `CERTIFICATE_VERIFY_FAILED` and the summary does not work with any key.

The certificate ships in the repository (`backend/certs/russian_trusted_root_ca.pem`), is baked into the image (`backend/Dockerfile` installs it both into the system store and into `/certs`) and is mounted into the container from `docker-compose.yml`. Nothing needs separate configuration: the code takes the file from `/certs`, and on a local run from `backend/certs/`. The `GIGACHAT_CA_BUNDLE` variable is needed only if the certificate lives in a non-standard place.

Disabling TLS verification (`GIGACHAT_VERIFY_SSL=false`) is a last resort for debugging: in a working contour verification is on.

Behaviour without a key and on failures

The feature is designed as optional: its failure must not break the main card workflow. In the demo contour (`GIGACHAT_FALLBACK_ENABLED=true`) a failure of the external service is not shown to the user as an error: the summary is assembled from the card data instead.

Situation	Response with `FALLBACK=false`	Response with `FALLBACK=true`
`GIGACHAT_CREDENTIALS` not set	503	200, demo summary
OAuth rejected the key	502	200, demo summary
GigaChat unreachable or 5xx	502	200, demo summary
Unexpected response format	502	200, demo summary
No access to the card	403	403

The demo summary is deterministic text assembled from the card fields (`gigachat.build_fallback_summary`): university, product, stage, assignee, the task counter and the last comment. It contains no generated claims, and the "Model" field of such a response reads `demo summary (GigaChat was not called)` — so demo text cannot be mistaken for a model answer.

A failure of the external service never turns into a 500: the response code shows that the problem is on the GigaChat side, not in the CRM. Data from someone else's card never leaves for the external service — access is checked before the prompt is assembled.

Code

- `backend/services/gigachat.py` — authorization, token cache, prompt assembly, response parsing, the `GigaChatDisabled` and `GigaChatUnavailable` error types, and the demo summary `build_fallback_summary`. The module is not called from `/readyz`: the unavailability of an optional feature must not put the service into a degraded state.
- `backend/api/interactions.py` — the endpoint `POST /api/interactions/{id}/summary`.
- `frontend/src/components/interaction/InteractionDrawer.tsx` — the "Summary" tab with a button and the generation time.

Tests

`tests/test_gigachat_summary.py` — 16 tests. The external service is replaced by an httpx transport: GigaChat is paid and unavailable in CI, and what is verified is our code, not someone else's service. Covered are prompt assembly, both authorization steps, the token cache, the 503 and 502 codes, the

denial of access to someone else's card and both demo summary modes — for a missing key and for an external service failure.